

Technical Documentation - Gaïa's Guardian

Version: Final (April 18, 2025) Team: Autimatique Project: Gaïa's Guardian

Table of Contents

1. Introduction.....1

2. Software Architecture.....2

3. Game Systems.....3

4. Design Patterns and Best Practices.....12

5. Asset Management.....28

6. Optimization and Performance.....34

7. Build System and Deployment.....47

8. Testing and Debugging.....54

9. Reference API.....63

10. Appendices.....70

1. Introduction

1.1 About Gaïa's Guardian

Gaïa's Guardian is a 2D action platformer where the player controls Gaïa, a mystical entity born from the remnants of the natural world, who fights corrupted machines to restore ecological balance. The game emphasizes fluid movement, rich combat mechanics, and dynamic environment evolution.

1.2 Documentation Objectives

This technical documentation aims to provide current and future developers with a complete understanding of the architecture, systems, and tools used in the development of Gaïa's Guardian. It will serve as a reference for maintenance, extension, and evolution of the project.

1.3 Technologies and Tools Used

- Game Engine: Unity 2023.2
- Programming Language: C#
- Version Control: Git, GitHub
- Project Management: Asana
- Communication: Slack
- Audio: FMOD
- Graphic Tools: Aseprite, Photoshop, Blender

2. Software Architecture

2.1 Overview

Gaïa's Guardian's architecture follows the MVC (Model-View-Controller) pattern adapted for video game development, with strong separation of responsibilities to ensure maintainability and scalability.

GaiaGuardian/

```
├── Assets/
|
|   ├── Scripts/
|   |
|   |   ├── Core/      # Core game system
|   |   ├── Controllers/ # Gameplay controllers
|   |   ├── Models/    # Data models
|   |   ├── Views/     # Display components
|   |   ├── Utils/     # Generic utilities
|   |   ├── AI/        # Artificial intelligence
|   |   ├── Audio/     # Audio management
|   |   └── UI/        # User interface
|
|   ├── Prefabs/      # Prefabricated objects
|
|   ├── Scenes/      # Game scenes
|
|   ├── Graphics/    # Graphic assets
|   |
|   |   ├── Characters/ # Character sprites
|   |   ├── Environment/ # Environment elements
|   |   ├── Animations/ # Animations
|   |   └── FX/        # Visual effects
|
|   ├── Audio/      # Audio assets
|   |
|   |   ├── Music/    # Music tracks
|   |   ├── SFX/     # Sound effects
|   |   └── FMOD/     # FMOD projects
|
|   └── Resources/   # Dynamically loaded resources
```

2.2 Main Class Diagram

Note: The complete UML diagram is available in Appendix A.

2.3 Data Flow

The data flow in Gaïa's Guardian follows an event-based communication model to minimize direct dependencies between components:

1. User inputs are captured by the InputManager
2. The PlayerController interprets these inputs and updates the PlayerModel
3. State changes trigger events
4. Subscribed systems (combat, animation, audio, etc.) react to these events
5. The GameManager coordinates high-level systems (progression, levels, etc.)

2.4 Game Initialization

Initialization sequence:

1. Loading essential services via ServiceLocator
2. Initialization of system managers (audio, save, etc.)
3. Loading progression data (if a save exists)
4. Initialization of current level and entities
5. Starting the game loop

3. Game Systems

3.1 Player Character System

3.1.1 Character Structure

csharp

```
public class PlayerController : MonoBehaviour
{
    [SerializeField] private PlayerModel _playerModel;
    [SerializeField] private PlayerView _playerView;

    private StateMachine _stateMachine;
    private MovementSystem _movementSystem;
    private CombatSystem _combatSystem;
    private AbilitySystem _abilitySystem;

    // Implementation...
}
```

Gaïa's character uses a modular structure:

- PlayerModel: Contains all state data and attributes
- PlayerView: Manages visual representation and animations

- PlayerController: Coordinates logic and systems

3.1.2 Movement System

Gaïa's movement system is based on a custom physics controller for better precision than Unity's standard system.

Main features:

- Capsule-shaped collision detection
- Special handling for platforms and surfaces
- Advanced input buffer system for jumps
- Management of movement states (air, ground, wall grab, etc.)

csharp

```
public class MovementSystem
{
    private PlayerModel _model;
    private Rigidbody2D _rigidbody;
    private PhysicsController _physicsController;

    // Configurable movement parameters
    public float MoveSpeed = 8f;
    public float JumpForce = 18f;
    public float DashDistance = 5f;
    public float WallSlideSpeed = 2f;

    // Implementation...
}
```

3.1.3 Combat System

The combat system manages Gaïa's attacks and interactions with enemies:

- Hitbox management for different attacks
- Damage and effect calculations
- Combo chains and timing
- Visual effects and feedback

csharp

```

public class CombatSystem
{
    private PlayerModel _model;
    private AttackDatabase _attackDatabase;

    // Attack and combo management
    private Dictionary<AttackType, Attack> _attacks;
    private ComboHandler _comboHandler;

    // Implementation...
}

```

3.1.4 Ability System

The ability system manages Gaïa's special powers:

- Progressive acquisition of new abilities
- Energy cost management
- Cooldowns and usage restrictions
- Environmental effects

```

csharp
public class AbilitySystem
{
    private PlayerModel _model;
    private Dictionary<AbilityType, Ability> _abilities;

    // Energy and cooldown management
    public float CurrentEnergy { get; private set; }
    private Dictionary<AbilityType, float> _cooldowns;

    // Implementation...
}

```

3.2 AI and Enemy System

3.2.1 AI Architecture

Enemy AI is based on a finite state machine (FSM) with modular behaviors:

csharp

```
public class EnemyController : MonoBehaviour
{
    [SerializeField] private EnemyModel _enemyModel;
    [SerializeField] private EnemyView _enemyView;

    private StateMachine _stateMachine;
    private Dictionary<Type, IEnemyBehavior> _behaviors;

    // Implementation...
}
```

Common state types:

- IdleState: Enemy is inactive
- PatrolState: Enemy patrols in a defined area
- AlertState: Enemy has detected the player but is not yet in combat
- ChaseState: Enemy actively pursues the player
- AttackState: Enemy executes an attack
- StunnedState: Enemy is temporarily neutralized
- DeadState: Enemy is defeated

3.2.2 Detection and Perception

The perception system allows enemies to detect Gaïa:

- Visual detection using raycasting
- Proximity detection using trigger zones
- Alert system with memory of last known position
- Communication between enemies (group alerts)

csharp

```
public class PerceptionSystem
{
    private EnemyModel _model;
    private Transform _target;
```

```

public bool CanSeeTarget { get; private set; }
public Vector2 LastKnownPosition { get; private set; }

// Detection methods

public bool CheckLineOfSight(Transform target, float viewDistance, float viewAngle)
{
    // Implementation...
}

// Implementation...
}

```

3.2.3 Pathfinding

The navigation system uses a combination of:

- Navigation grid for global pathfinding
- Raycast for local navigation and obstacle avoidance
- Waypoints for predefined patrol paths

csharp

```

public class NavigationSystem
{
    private EnemyModel _model;
    private GridPathfinder _pathfinder;

    private List<Vector2> _currentPath;
    private int _currentWaypointIndex;

    // Implementation...

    public List<Vector2> FindPath(Vector2 start, Vector2 end)
    {
        // Pathfinding implementation...
    }
}

```

```
}  
}
```

3.2.4 Specific Enemy Types

Each enemy type inherits from a base class but implements specific behaviors:

csharp

// Abstract base for all enemies

```
public abstract class Enemy : MonoBehaviour
```

```
{
```

// Common properties

// Virtual methods to implement

```
protected virtual void InitializeStateMachine() { }
```

```
protected virtual void RegisterBehaviors() { }
```

```
protected virtual void OnDamage(DamageInfo damageInfo) { }
```

```
}
```

// Specific implementations

```
public class DroneEnemy : Enemy
```

```
{
```

// Specific implementations...

```
}
```

```
public class SentinelEnemy : Enemy
```

```
{
```

// Specific implementations...

```
}
```

```
public class BossEnemy : Enemy
```

```
{
```

// Additional systems for boss phases

```
private BossPhaseManager _phaseManager;
```



```
// Specific implementations...
```

```
}
```

3.3 Level and Environment System

3.3.1 Level Structure

Gaïa's Guardian levels are structured using:

- Unity Tilemaps for basic elements
- Interactive objects instantiated as prefabs
- Triggers for event zones
- Portal system for connections between levels

```
csharp
```

```
public class LevelManager : MonoBehaviour
```

```
{
```

```
// Level component references
```

```
[SerializeField] private Transform _playerSpawnPoint;
```

```
[SerializeField] private List<GameObject> _enemySpawnPoints;
```

```
[SerializeField] private List<EnvironmentController> _environmentControllers;
```

```
// Level state management
```

```
private Dictionary<string, bool> _checkpoints;
```

```
private float _purificationLevel;
```

```
// Implementation...
```

```
}
```

3.3.2 Purification System

The purification system dynamically transforms the environment:

```
csharp
```

```
public class PurificationSystem : MonoBehaviour
```

```
{
```

```
// Purification zones
```

```
[SerializeField] private List<PurificationZone> _purificationZones;
```

```

// Global purification state

private float _globalPurificationLevel;

private Dictionary<string, float> _zonePurificationLevels;


// Events

public event Action<float> OnGlobalPurificationChanged;

public event Action<string, float> OnZonePurificationChanged;


// Implementation...
}

```

Purification affects:

- Visual appearance (via transition shaders)
- Scenery objects (asset transformation)
- Enemy behaviors (weakening)
- Accessibility of certain areas

3.3.3 Interaction with Environment Elements

Interactive objects follow a common interface:

```

csharp

public interface IInteractable
{
    bool CanInteract(PlayerController player);
    void OnInteract(PlayerController player);
    void DisplayInteractionPrompt(bool display);
}

```

Types of interactive objects:

- PurificationNode: Purification points
- Collectible: Objects to collect
- PlatformSwitch: Switches activating platforms
- EnvironmentHazard: Traps and dangers
- ClimbableWall: Climbing surfaces

3.4 Progression System

3.4.1 Ability Unlocking

```
csharp

public class ProgressionManager : MonoBehaviour
{
    // Unlocked abilities

    private HashSet<AbilityType> _unlockedAbilities;

    // Player progression

    private int _collectedEssence;
    private Dictionary<string, bool> _completedSanctuaries;

    // Implementation...

    public void UnlockAbility(AbilityType abilityType)
    {
        if (_unlockedAbilities.Add(abilityType))
        {
            // Notify and save...
        }
    }
}
```

3.4.2 Save System

The save system uses JSON format to store:

- Player progression (abilities, collectibles)
- World state (purified zones, completed sanctuaries)
- Current position and checkpoint

```
csharp

public class SaveSystem : MonoBehaviour
{
    private const string SAVE_FILE_NAME = "gaia_save.json";
```

```

public SaveData CurrentSave { get; private set; }

// Implementation...

public void SaveGame()
{
    // Get current data

    SaveData saveData = new SaveData
    {
        // Fill with current data...
    };

    // Serialize and save

    string json = JsonUtility.ToJson(saveData);
    File.WriteAllText(GetSavePath(), json);
}

public bool LoadGame()
{
    string path = GetSavePath();
    if (File.Exists(path))
    {
        string json = File.ReadAllText(path);

        CurrentSave = JsonUtility.FromJson<SaveData>(json);

        return true;
    }

    return false;
}
}

```

4. Design Patterns and Best Practices

4.1 Implemented Patterns

4.1.1 MVC (Model-View-Controller)

Used for the main entities:

csharp

// Model - Data

```
public class PlayerModel
```

```
{
```

```
    // Player attributes
```

```
    public float Health { get; private set; }
```

```
    public float Energy { get; private set; }
```

```
    public Vector2 Velocity { get; set; }
```

```
    // Current state
```

```
    public PlayerState CurrentState { get; set; }
```

```
    // Events
```

```
    public event Action<float> OnHealthChanged;
```

```
    public event Action<float> OnEnergyChanged;
```

```
    public event Action<PlayerState> OnStateChanged;
```

```
    // Implementation...
```

```
}
```

// View - Visual representation

```
public class PlayerView : MonoBehaviour
```

```
{
```

```
    // Graphic components
```

```
    [SerializeField] private Animator _animator;
```

```
    [SerializeField] private SpriteRenderer _renderer;
```

```
    [SerializeField] private ParticleSystem _moveParticles;
```

```

// Animation and display methods

public void UpdateAnimation(PlayerState state, Vector2 velocity)
{
    // Update animation parameters...
}

// Implementation...
}

```

```

// Controller - Logic

public class PlayerController : MonoBehaviour
{
    [SerializeField] private PlayerModel _model;
    [SerializeField] private PlayerView _view;

    // Systems

    private MovementSystem _movement;
    private CombatSystem _combat;

    // Implementation...
}

```

4.1.2 State Machine

Used for player and enemy state management:

```

csharp

public class StateMachine
{
    private IState _currentState;
    private Dictionary<Type, IState> _states = new Dictionary<Type, IState>();

    public void RegisterState(IState state)
    {

```

```
    _states[state.GetType()] = state;
}
```

```
public void ChangeState<T>() where T : IState
{
    if (_currentState != null)
    {
        _currentState.Exit();
    }
}
```

```
    _currentState = _states[typeof(T)];
    _currentState.Enter();
}
```

```
public void Update()
{
    _currentState?.Update();
}
```

```
public void FixedUpdate()
{
    _currentState?.FixedUpdate();
}
}
```

```
public interface IState
{
    void Enter();
    void Update();
    void FixedUpdate();
    void Exit();
}
```

```
}
```

4.1.3 Event-Driven Programming

Using events to decouple systems:

csharp

```
// Centralized event system
```

```
public static class EventManager
```

```
{
```

```
// Generic event dictionary
```

```
private static Dictionary<Type, Delegate> _events = new Dictionary<Type, Delegate>();
```

```
// Subscription and trigger methods
```

```
public static void Subscribe<T>(Action<T> listener) where T : struct
```

```
{
```

```
// Implementation...
```

```
}
```

```
public static void Unsubscribe<T>(Action<T> listener) where T : struct
```

```
{
```

```
// Implementation...
```

```
}
```

```
public static void Trigger<T>(T eventData) where T : struct
```

```
{
```

```
// Implementation...
```

```
}
```

```
}
```

```
// Usage example
```

```
public struct PlayerHealthChangedEvent
```

```
{
```

```
public float CurrentHealth;
```



```

    public float PreviousHealth;
}

// In a subscribed class
void OnEnable()
{
    EventManager.Subscribe<PlayerHealthChangedEvent>(OnPlayerHealthChanged);
}

void OnDisable()
{
    EventManager.Unsubscribe<PlayerHealthChangedEvent>(OnPlayerHealthChanged);
}

void OnPlayerHealthChanged(PlayerHealthChangedEvent evt)
{
    // React to the change...
}

```

4.1.4 Singleton

Used sparingly for global managers:

csharp

```
public class GameManager : MonoBehaviour
```

```
{
    private static GameManager _instance;
```

```
    public static GameManager Instance
```

```
{
    get
    {
        if (_instance == null)
        {
```

```

        Debug.LogError("GameManager not found in scene");
    }
    return _instance;
}
}

```

```

private void Awake()
{
    if (_instance != null && _instance != this)
    {
        Destroy(gameObject);
        return;
    }

```

```

    _instance = this;
    DontDestroyOnLoad(gameObject);
}

```

```

// Implementation...
}

```

4.1.5 Object Pooling

Used to optimize frequent object creation/destruction:

csharp

```

public class ObjectPool<T> where T : Component

```

```

{
    private T _prefab;
    private List<T> _inactiveObjects = new List<T>();

```

```

    public ObjectPool(T prefab, int initialSize)

```

```

    {
        _prefab = prefab;

```

```

    for (int i = 0; i < initialSize; i++)
    {
        T obj = Object.Instantiate(_prefab);
        obj.gameObject.SetActive(false);
        _inactiveObjects.Add(obj);
    }
}

```

```

public T Get()
{
    if (_inactiveObjects.Count > 0)
    {
        T obj = _inactiveObjects[0];
        _inactiveObjects.RemoveAt(0);
        obj.gameObject.SetActive(true);
        return obj;
    }

    return Object.Instantiate(_prefab);
}

```

```

public void Return(T obj)
{
    obj.gameObject.SetActive(false);
    _inactiveObjects.Add(obj);
}
}

```

4.2 SOLID Principles

4.2.1 Single Responsibility Principle (S)

Each class has a single, well-defined responsibility:

csharp

// Bad example: class with too many responsibilities

```
public class BadPlayer : MonoBehaviour
{
    // Handles input, movement, combat, animations, etc.
    // ...
}
```

// Good example: separated responsibilities

```
public class InputHandler : MonoBehaviour { /* ... */ }
public class MovementSystem : MonoBehaviour { /* ... */ }
public class CombatSystem : MonoBehaviour { /* ... */ }
public class AnimationController : MonoBehaviour { /* ... */ }
```

4.2.2 Open/Closed Principle (O)

Classes are open for extension but closed for modification:

csharp

// Base class for powers

```
public abstract class Ability
{
    public abstract string Name { get; }
    public abstract float EnergyCost { get; }
    public abstract float Cooldown { get; }

    public abstract void Activate(PlayerController player);
}
```

// Extensions without modifying the base class

```
public class DashAbility : Ability
{
    public override string Name => "Dash";
    public override float EnergyCost => 10f;
}
```

```

public override float Cooldown => 2f;

public override void Activate(PlayerController player)
{
    // Specific implementation...
}
}

```

```

public class RootWallAbility : Ability
{
    public override string Name => "Root Wall";
    public override float EnergyCost => 25f;
    public override float Cooldown => 5f;

    public override void Activate(PlayerController player)
    {
        // Specific implementation...
    }
}

```

4.2.3 Liskov Substitution Principle (L)

Derived classes can replace their base classes:

```

csharp
public abstract class Enemy
{
    public abstract void TakeDamage(float damage);
    public abstract void Attack(PlayerController target);
}

```

```

public class DroneEnemy : Enemy
{
    public override void TakeDamage(float damage)

```

```

{
    // Specific implementation...
}

public override void Attack(PlayerController target)
{
    // Ranged attack...
}
}

public class SentinelEnemy : Enemy
{
    public override void TakeDamage(float damage)
    {
        // Specific implementation...
    }

    public override void Attack(PlayerController target)
    {
        // Melee attack...
    }
}

// Polymorphic usage
public void DamageAllEnemies(List<Enemy> enemies, float damage)
{
    foreach (Enemy enemy in enemies)
    {
        enemy.TakeDamage(damage); // Works for all enemy types
    }
}

```

4.2.4 Interface Segregation Principle (I)

Specific interfaces rather than monolithic ones:

csharp

// Bad example: too large interface

```
public interface IBadGameObject
{
    void Update();
    void Render();
    void TakeDamage(float damage);
    void Attack(IDamageable target);
    void Move(Vector2 direction);
    void Jump();
    // ...
}
```

// Good example: interfaces separated by role

```
public interface IUpdatable
{
    void Update();
}
```

```
public interface IRenderable
{
    void Render();
}
```

```
public interface IDamageable
{
    void TakeDamage(float damage);
}
```

```
public interface IAttacker
{
    void Attack(IDamageable target);
}
```

```
public interface IMovable
{
    void Move(Vector2 direction);
}
```

// A class implements only the interfaces it needs

```
public class Enemy : IUpdatable, IRenderable, IDamageable, IAttacker, IMovable
{
    // Implementations...
}
```

```
public class Projectile : IUpdatable, IRenderable
{
    // Implementations...
}
```

4.2.5 Dependency Inversion Principle (D)

Dependencies on abstractions, not implementations:

csharp

// Bad example: direct dependency

```
public class BadPlayerController
{
    private AudioManager _audioManager; // Concrete dependency

    public BadPlayerController()
    {
        _audioManager = new AudioManager(); // Strong coupling
    }
}
```



```

    }

    public void Jump()
    {
        // Jump logic...
        _audioManager.PlaySound("jump"); // Direct call
    }
}

// Good example: dependency inversion
public interface IAudioService
{
    void PlaySound(string soundId);
}

public class AudioManager : IAudioService
{
    public void PlaySound(string soundId)
    {
        // Implementation...
    }
}

public class PlayerController
{
    private IAudioService _audioService; // Abstract dependency

    public PlayerController(IAudioService audioService)
    {
        _audioService = audioService; // Dependency injection
    }
}

```

```

public void Jump()
{
    // Jump logic...

    _audioService.PlaySound("jump"); // Call via interface
}
}

```

4.3 Code Conventions

4.3.1 Naming Conventions

- Classes and methods: PascalCase
- Variables and parameters: camelCase
- Constants: UPPER_SNAKE_CASE
- Private fields: _camelCase (prefixed with underscore)
- Interfaces: IPascalCase (prefixed with 'I')
- Enumerations: PascalCase

csharp

```

public class PlayerController
{
    private const float MAX_HEALTH = 100f;

    [SerializeField] private float _moveSpeed;
    private Vector2 _currentVelocity;

    public float Health { get; private set; }

    public void ApplyDamage(float damageAmount)
    {
        // Implementation...
    }
}

```

4.3.2 Code Organization

- One file per class
- Related classes grouped in namespaces
- Order of elements in a class:
 1. Private fields
 2. Public properties
 3. Constructors
 4. Unity methods (Awake, Start, Update, etc.)
 5. Public methods
 6. Private methods

csharp

namespace GaiaGuardian.Player

{

public class PlayerController : MonoBehaviour

{

// Private fields

[SerializeField] private float _moveSpeed;

private Vector2 _moveInput;

// Public properties

public bool IsGrounded { get; private set; }

public PlayerState CurrentState { get; private set; }

// Constructor (if applicable for ScriptableObjects)

// Unity methods

private void Awake()

{

// Initialization...

}

private void Start()

```

{
    // Setup...
}

private void Update()
{
    // Update logic...
}

// Public methods
public void Jump()
{
    // Implementation...
}

public void ApplyDamage(float amount)
{
    // Implementation...
}

// Private methods
private void CheckGrounded()
{
    // Implementation...
}
}

```

5. Asset Management

5.1 Structure and Organization

5.1.1 Folder Hierarchy

Assets/

```

└─ Prefabs/      # Prefabricated objects
| └─ Player/     # Player-related prefabs
| └─ Enemies/    # Enemy prefabs
| └─ Environment/ # Environment elements
| └─ UI/         # Interface elements
| └─ FX/         # Prefabricated visual effects
└─ Graphics/     # Graphic assets
| └─ Characters/ # Character sprites
| | └─ Gaia/     # Main player
| | └─ Drones/   # Drone enemies
| | └─ Sentinels/ # Mechanical sentinels
| | └─ Bosses/   # End-level bosses
| └─ Environment/ # Scenery and environment
| | └─ Tilemaps/  # Tile sets for levels
| | └─ Props/     # Decorative objects
| | └─ Background/ # Parallax backgrounds
| | └─ Foreground/ # Foreground elements
| └─ UI/         # User interface elements
| └─ FX/         # Visual effects
└─ Animations/   # Animation controllers and clips
└─ Audio/       # Audio assets
| └─ Music/      # Music tracks
| └─ SFX/       # Sound effects
| └─ FMOD/      # FMOD projects
└─ Resources/   # Dynamically loaded resources

```

5.1.2 Naming Conventions

To maintain consistency and facilitate teamwork, we follow these naming conventions:

- **Descriptive prefixes** for all files:
 - PL_ - Player-related assets
 - EN_ - Enemy-related assets

- ENV_ - Environment assets
- UI_ - User interface elements
- FX_ - Visual effects
- SFX_ - Sound effects
- **Functional suffixes** indicating asset type:
 - _SPR - Sprite
 - _ANM - Animation
 - _PFB - Prefab
 - _MAT - Material
 - _SHD - Shader
 - _SND - Sound

Example: PL_Dash_ANM (Player dash animation)

5.2 Graphic Asset Pipeline

5.2.1 Aseprite to Unity Workflow

1. Create sprites and animations in Aseprite
2. Export in PNG format using automatic export script
3. Import into Unity with the following settings:
 - Pixel Per Unit: 16
 - Filter Mode: Point (no filter)
 - Compression: None
4. Sprite Sheet configuration:
 - Mode: Multiple
 - Pivot: Bottom Center for characters
 - Pivot: Center for objects

csharp

// Example of automated import utility

```
public static class AsepriteImporter
{
    [MenuItem("Tools/Import Aseprite Files")]
    public static void ImportAsepriteFiles()
    {
```

```

// Locate all .ase files in the import folder

string[] asepritePaths = Directory.GetFiles(
    Application.dataPath + "/AsepriteSources",
    "*.ase",
    SearchOption.AllDirectories
);

foreach (string asePath in asepritePaths)
{
    // Execute export via Aseprite CLI
    ExportSpritesFromAseprite(asePath);

    // Configure the sprite in Unity
    ConfigureImportedSprite(asePath);
}
}

// Implementation of helper methods...
}

```

5.2.2 Sprite Atlas Management

To optimize rendering performance, we use sprite atlases generated by Unity:

```

csharp
public class AtlasGenerator : MonoBehaviour
{
    [Serializable]
    public class AtlasDefinition
    {
        public string name;
        public List<Sprite> sprites;
        public int maxSize = 2048;
    }
}

```

```
[SerializeField] private List<AtlasDefinition> _atlasDefinitions;
```

```
[MenuItem("Tools/Generate Sprite Atlases")]
```

```
public static void GenerateAtlases()
```

```
{
```

```
    // Atlas generation implementation...
```

```
}
```

```
}
```

5.3 Audio Management with FMOD

5.3.1 FMOD Project Structure

The FMOD project is organized into banks:

- Master - Global mixing and main audio parameters
- Music - Music tracks with transitions between purification states
- SFX_Player - Sounds related to player actions
- SFX_Enemies - Sounds related to enemies
- SFX_Environment - Ambient and environment sounds
- UI - User interface sounds

5.3.2 Unity-FMOD Integration

```
csharp
```

```
public class AudioManager : MonoBehaviour
```

```
{
```

```
    private static AudioManager _instance;
```

```
    public static AudioManager Instance => _instance;
```

```
    // FMOD parameters for dynamic audio control
```

```
    private FMOD.Studio.EventInstance _musicInstance;
```

```
    private FMOD.Studio.EventInstance _ambienceInstance;
```

```
    // Purification level affecting music (0-100)
```



```

private float _purificationLevel = 0f;

private void Awake()
{
    if (_instance != null && _instance != this)
    {
        Destroy(gameObject);
        return;
    }

    _instance = this;
    DontDestroyOnLoad(gameObject);

    InitializeAudio();
}

private void InitializeAudio()
{
    _musicInstance = FMODUnity.RuntimeManager.CreateInstance("event:/Music/LevelMusic");
    _ambienceInstance =
FMODUnity.RuntimeManager.CreateInstance("event:/SFX_Environment/Ambiance");

    _musicInstance.start();
    _ambienceInstance.start();
}

// Update parameters based on game state
public void UpdatePurificationLevel(float level)
{
    _purificationLevel = Mathf.Clamp(level, 0f, 100f);
    _musicInstance.setParameterByName("Purification", _purificationLevel);
}

```

```

        _ambienceInstance.setParameterByName("Purification", _purificationLevel);
    }

    // Play a specific sound
    public void PlaySound(string eventPath)
    {
        FMODUnity.RuntimeManager.PlayOneShot(eventPath);
    }

    // Play a sound at a spatial position
    public void PlaySound3D(string eventPath, Vector3 position)
    {
        FMODUnity.RuntimeManager.PlayOneShot(eventPath, position);
    }

    // Cleanup
    private void OnDestroy()
    {
        _musicInstance.stop(FMOD.Studio.STOP_MODE.ALLOWFADEOUT);
        _ambienceInstance.stop(FMOD.Studio.STOP_MODE.ALLOWFADEOUT);
        _musicInstance.release();
        _ambienceInstance.release();
    }
}

```

6. Optimization and Performance

6.1 General Optimization Principles

6.1.1 Performance Monitoring

We use the Unity Profiler to monitor:

- CPU usage
- Memory allocations
- Rendering time

- Physics load

Performance targets for all platforms:

- Consistent 60 FPS
- CPU time per frame < 16ms
- Drawcalls < 100 per frame
- Memory usage < 500MB

6.1.2 Critical Code Optimizations

csharp

// Bad example: expensive checks in Update

```
private void BadUpdate()
{
    // Expensive calculation every frame
    foreach (GameObject enemy in GameObject.FindGameObjectsWithTag("Enemy"))
    {
        float distance = Vector3.Distance(transform.position, enemy.transform.position);
        if (distance < detectionRadius)
        {
            // Processing...
        }
    }
}
```

// Good example: optimized

```
private List<Enemy> _enemiesInProximity = new List<Enemy>();
private float _updateTimer = 0f;
private const float UPDATE_INTERVAL = 0.25f; // Check only 4 times per second
private void GoodUpdate()
{
    _updateTimer += Time.deltaTime;

    if (_updateTimer >= UPDATE_INTERVAL)
```

```

{
    _updateTimer = 0f;
    UpdateEnemiesInProximity();
}

// Using the pre-filtered list
foreach (Enemy enemy in _enemiesInProximity)
{
    // More efficient processing...
}
}

private void UpdateEnemiesInProximity()
{
    _enemiesInProximity.Clear();

    Collider2D[] colliders = Physics2D.OverlapCircleAll(transform.position, detectionRadius,
    enemyLayerMask);

    foreach (Collider2D collider in colliders)
    {
        if (collider.TryGetComponent<Enemy>(out Enemy enemy))
        {
            _enemiesInProximity.Add(enemy);
        }
    }
}

```

6.2 Rendering Optimization

6.2.1 Batching and Sprite Atlases

To maximize batching:

- All terrain tiles use the same material
- Sprites are grouped into atlases by level/zone

- Static hierarchy is used for immobile objects

csharp

```
public class SpriteOptimizer : MonoBehaviour
{
    [SerializeField] private Transform _containerTransform;

    [ContextMenu("Optimize Static Sprites")]
    public void OptimizeStaticSprites()
    {
        SpriteRenderer[] renderers =
            _containerTransform.GetComponentsInChildren<SpriteRenderer>();

        Dictionary<Material, List<SpriteRenderer>> materialGroups = new Dictionary<Material,
            List<SpriteRenderer>>();

        // Group by material
        foreach (SpriteRenderer renderer in renderers)
        {
            if (!materialGroups.ContainsKey(renderer.material))
            {
                materialGroups[renderer.material] = new List<SpriteRenderer>();
            }

            materialGroups[renderer.material].Add(renderer);

            // Mark as static for batching
            renderer.gameObject.isStatic = true;
        }

        Debug.Log($"Optimized {renderers.Length} sprites into {materialGroups.Count} batches");
    }
}
```

6.2.2 Occlusion and Culling

csharp

```
public class DynamicObjectCulling : MonoBehaviour
{
    [SerializeField] private float _cullDistance = 20f;
    [SerializeField] private Transform _playerTransform;

    private List<CullableObject> _cullableObjects = new List<CullableObject>();

    [System.Serializable]
    private class CullableObject
    {
        public Transform transform;
        public Behaviour[] componentsToDisable;
        public bool isActive = true;
    }

    private void Start()
    {
        RegisterCullableObjects();
    }

    private void Update()
    {
        UpdateCulling();
    }

    private void RegisterCullableObjects()
    {
        // Find all objects with the "Cullable" tag
        foreach (GameObject obj in GameObject.FindGameObjectsWithTag("Cullable"))
```

```

{
    CullableObject cullable = new CullableObject
    {
        transform = obj.transform,
        componentsToDisable = obj.GetComponents<Behaviour>(),
        isActive = true
    };

    _cullableObjects.Add(cullable);
}

private void UpdateCulling()
{
    foreach (CullableObject cullable in _cullableObjects)
    {
        float distance = Vector3.Distance(_playerTransform.position, cullable.transform.position);
        bool shouldBeActive = distance <= _cullDistance;

        if (shouldBeActive != cullable.isActive)
        {
            SetObjectActive(cullable, shouldBeActive);
        }
    }
}

private void SetObjectActive(CullableObject cullable, bool active)
{
    foreach (Behaviour component in cullable.componentsToDisable)
    {
        component.enabled = active;
    }
}

```

```

    }

    cullable.isActive = active;
}
}

```

6.3 Physics Optimization

6.3.1 Collider and Layer Settings

To minimize physics calculations:

- Use of simplified collision shapes
- Appropriate configuration of layer collision matrix
- Code-based collision management for complex objects

csharp

// Configuration of the collision matrix in code

```
public class PhysicsLayerManager : MonoBehaviour
```

```
{
```

```
    private void Awake()
```

```
    {
```

```
        ConfigureCollisionMatrix();
```

```
    }
```

```
    private void ConfigureCollisionMatrix()
```

```
    {
```

```
        // Configure layers that don't interact
```

```
        Physics2D.IgnoreLayerCollision(LayerMask.NameToLayer("Player"),
        LayerMask.NameToLayer("PlayerProjectile"), true);
```

```
        Physics2D.IgnoreLayerCollision(LayerMask.NameToLayer("Enemy"),
        LayerMask.NameToLayer("EnemyProjectile"), true);
```

```
        Physics2D.IgnoreLayerCollision(LayerMask.NameToLayer("PlayerProjectile"),
        LayerMask.NameToLayer("PlayerProjectile"), true);
```

```
        Physics2D.IgnoreLayerCollision(LayerMask.NameToLayer("EnemyProjectile"),
        LayerMask.NameToLayer("EnemyProjectile"), true);
```

```
    }
```



```
}
```

6.3.2 Optimized Raycasting

```
csharp
```

```
public class OptimizedGroundCheck : MonoBehaviour
```

```
{
```

```
    [SerializeField] private LayerMask _groundMask;
```

```
    [SerializeField] private Transform _feet;
```

```
    [SerializeField] private float _rayDistance = 0.2f;
```

```
    // Result caching
```

```
    private RaycastHit2D[] _raycastHits = new RaycastHit2D[4];
```

```
    private ContactFilter2D _contactFilter;
```

```
    private void Awake()
```

```
{
```

```
        // Configure contact filter only once
```

```
        _contactFilter = new ContactFilter2D
```

```
{
```

```
    useLayerMask = true,
```

```
    layerMask = _groundMask,
```

```
    useTriggers = false
```

```
};
```

```
}
```

```
    public bool CheckIsGrounded()
```

```
{
```

```
        // Single raycast with multiple results
```

```
        int hitCount = Physics2D.Raycast(_feet.position, Vector2.down, _contactFilter, _raycastHits, _rayDistance);
```

```
        return hitCount > 0;
```

```
}
```

```
}
```

6.4 Memory Management

6.4.1 Object Pooling

We use an extensive pooling system for all frequently created/destroyed objects:

- Projectiles
- Particle effects
- Enemies
- Collectibles

```
csharp
```

```
// Centralized pool manager
```

```
public class PoolManager : MonoBehaviour
```

```
{
```

```
    private static PoolManager _instance;
```

```
    public static PoolManager Instance => _instance;
```

```
[System.Serializable]
```

```
public class Pool
```

```
{
```

```
    public string tag;
```

```
    public GameObject prefab;
```

```
    public int size;
```

```
}
```

```
[SerializeField] private List<Pool> _pools;
```

```
private Dictionary<string, Queue<GameObject>> _poolDictionary;
```

```
private void Awake()
```

```
{
```

```
    _instance = this;
```

```

_poolDictionary = new Dictionary<string, Queue<GameObject>>();

foreach (Pool pool in _pools)
{
    Queue<GameObject> objectPool = new Queue<GameObject>();

    for (int i = 0; i < pool.size; i++)
    {
        GameObject obj = Instantiate(pool.prefab);
        obj.SetActive(false);
        objectPool.Enqueue(obj);
    }

    _poolDictionary.Add(pool.tag, objectPool);
}

}

public GameObject SpawnFromPool(string tag, Vector3 position, Quaternion rotation)
{
    if (!_poolDictionary.ContainsKey(tag))
    {
        Debug.LogWarning($"Pool with tag {tag} doesn't exist.");
        return null;
    }

    GameObject objectToSpawn = _poolDictionary[tag].Count > 0
        ? _poolDictionary[tag].Dequeue()
        : Instantiate(_pools.Find(p => p.tag == tag).prefab);

    objectToSpawn.SetActive(true);
    objectToSpawn.transform.position = position;
}

```

```
objectToSpawn.transform.rotation = rotation;
```

```
IPoolable poolable = objectToSpawn.GetComponent<IPoolable>();
```

```
if (poolable != null)
```

```
{
```

```
    poolable.OnSpawnFromPool();
```

```
}
```

```
return objectToSpawn;
```

```
}
```

```
public void ReturnToPool(string tag, GameObject objectToReturn)
```

```
{
```

```
    if (!_poolDictionary.ContainsKey(tag))
```

```
    {
```

```
        Debug.LogWarning($"Pool with tag {tag} doesn't exist.");
```

```
        return;
```

```
    }
```

```
objectToReturn.SetActive(false);
```

```
IPoolable poolable = objectToReturn.GetComponent<IPoolable>();
```

```
if (poolable != null)
```

```
{
```

```
    poolable.OnReturnToPool();
```

```
}
```

```
_poolDictionary[tag].Enqueue(objectToReturn);
```

```
}
```

```
}
```

// Interface for poolable objects

```
public interface IPoolable
{
    void OnSpawnFromPool();
    void OnReturnToPool();
}
```

6.4.2 Resource Management

Strategies to reduce memory footprint:

- Asynchronous loading of scenes and resources
- Unloading unused resources
- Texture compression appropriate for each use case

csharp

```
public class ResourceManager : MonoBehaviour
{
    private Dictionary<string, UnityEngine.Object> _loadedResources = new Dictionary<string,
    UnityEngine.Object>();

    public T LoadResource<T>(string path) where T : UnityEngine.Object
    {
        string key = $"{typeof(T).Name}_{path}";

        if (_loadedResources.ContainsKey(key))
        {
            return _loadedResources[key] as T;
        }

        T resource = Resources.Load<T>(path);
        if (resource != null)
        {
            _loadedResources[key] = resource;
        }
    }
}
```

```

        return resource;
    }

    public void UnloadUnusedResources()
    {
        List<string> keysToRemove = new List<string>();

        foreach (var kvp in _loadedResources)
        {
            // Check if resource is still being used
            // Type-specific logic...

            if (!IsResourceInUse(kvp.Value))
            {
                keysToRemove.Add(kvp.Key);
            }
        }

        foreach (string key in keysToRemove)
        {
            _loadedResources.Remove(key);
        }

        // Force memory cleanup
        Resources.UnloadUnusedAssets();
        System.GC.Collect();
    }

    private bool IsResourceInUse(UnityEngine.Object resource)
    {

```

```

// Type-specific checking logic

// Simple example for a GameObject prefab
if (resource is GameObject)
{
    return GameObject.FindObjectsOfType<Transform>().Any(t =>
t.name.Contains(resource.name));
}

// Default, consider as used
return true;
}
}

```

7. Build System and Deployment

7.1 Build Configuration

7.1.1 Build Parameters

```

csharp
public class BuildManager : EditorWindow
{
    private BuildPlayerOptions _buildPlayerOptions;
    private BuildTarget _selectedTarget;
    private string _buildPath;

    [MenuItem("Tools/Build Manager")]
    public static void ShowWindow()
    {
        GetWindow<BuildManager>("Build Manager");
    }

    private void OnGUI()
    {
        GUILayout.Label("Gaia's Guardian Build Manager", EditorStyles.boldLabel);
    }
}

```

```
EditorGUILayout.Space();
```

```
_selectedTarget = (BuildTarget)EditorGUILayout.EnumPopup("Build Target", _selectedTarget);
```

```
_buildPath = EditorGUILayout.TextField("Build Path", _buildPath);
```

```
EditorGUILayout.Space();
```

```
if (GUILayout.Button("Build Development"))
```

```
{
```

```
    BuildGame(BuildOptions.Development);
```

```
}
```

```
if (GUILayout.Button("Build Release"))
```

```
{
```

```
    BuildGame(BuildOptions.None);
```

```
}
```

```
}
```

```
private void BuildGame(BuildOptions options)
```

```
{
```

```
    _buildPlayerOptions = new BuildPlayerOptions
```

```
{
```

```
        scenes = GetEnabledScenes(),
```

```
        locationPathName = _buildPath,
```

```
        target = _selectedTarget,
```

```
        options = options
```

```
};
```

```
BuildPipeline.BuildPlayer(_buildPlayerOptions);
```

```
}
```



```

private string[] GetEnabledScenes()
{
    List<string> sceneList = new List<string>();
    foreach (EditorBuildSettingsScene scene in EditorBuildSettings.scenes)
    {
        if (scene.enabled)
        {
            sceneList.Add(scene.path);
        }
    }
    return sceneList.ToArray();
}
}

```

7.1.2 Cross-Platform Configuration

We use conditional definitions to handle differences between platforms:

csharp

```

public class PlatformSpecificManager : MonoBehaviour
{
    // Platform detection

    public static bool IsMobile
    {
        get
        {
            #if UNITY_ANDROID || UNITY_IOS
                return true;
            #else
                return false;
            #endif
        }
    }
}

```

// Platform-specific initialization

private void Awake()

{

 ApplyPlatformSpecificSettings();

}

private void ApplyPlatformSpecificSettings()

{

 #if UNITY_STANDALONE

// PC configurations

 QualitySettings.vSyncCount = 1;

 Application.targetFrameRate = 60;

 #elif UNITY_ANDROID || UNITY_IOS

// Mobile configurations

 QualitySettings.vSyncCount = 0;

 Application.targetFrameRate = 30;

// Reduce effects quality

 ReduceEffectsQuality();

 #endif

}

private void ReduceEffectsQuality()

{

// Reduce particle effects quality for mobile

 ParticleSystem[] systems = FindObjectsOfType<ParticleSystem>();

 foreach (ParticleSystem system in systems)

 {

 var main = system.main;

 main.maxParticles /= 2;

 }

```
}  
}
```

7.2 Continuous Integration (CI/CD)

7.2.1 GitHub Actions Pipeline

We use GitHub Actions to automate:

- Build compilation
- Unit test execution
- Automatic version deployment

yaml

```
# .github/workflows/build.yml
```

```
name: Build Gaïa's Guardian
```

```
on:
```

```
  push:
```

```
    branches: [ main, develop ]
```

```
  pull_request:
```

```
    branches: [ main, develop ]
```

```
jobs:
```

```
  buildForAllSupportedPlatforms:
```

```
    name: Build for ${{ matrix.targetPlatform }}
```

```
    runs-on: ubuntu-latest
```

```
    strategy:
```

```
      fail-fast: false
```

```
      matrix:
```

```
        targetPlatform:
```

- StandaloneWindows64
- StandaloneLinux64
- StandaloneOSX

```
    steps:
```

- uses: actions/checkout@v2

with:

fetch-depth: 0

lfs: true

- uses: actions/cache@v2

with:

path: Library

key: Library-`{{ matrix.targetPlatform }}`-`{{ hashFiles('Assets/**', 'Packages/**', 'ProjectSettings/**') }}`

restore-keys: |

Library-`{{ matrix.targetPlatform }}`-

- uses: game-ci/unity-builder@v2

env:

UNITY_LICENSE: `{{ secrets.UNITY_LICENSE }}`

with:

targetPlatform: `{{ matrix.targetPlatform }}`

- uses: actions/upload-artifact@v2

with:

name: Build-`{{ matrix.targetPlatform }}`

path: build/`{{ matrix.targetPlatform }}`

7.2.2 Automated Tests

Automated tests are executed on each push:

yml

`# .github/workflows/test.yml`

name: Test Gaia's Guardian

on:

push:

branches: [main, develop]

pull_request:

branches: [main, develop]

jobs:

testAllModes:

name: Test in \${{ matrix.testMode }}

runs-on: ubuntu-latest

strategy:

fail-fast: false

matrix:

testMode:

- playmode

- editmode

steps:

- uses: actions/checkout@v2

with:

lfs: true

- uses: actions/cache@v2

with:

path: Library

key: Library-\${{ matrix.testMode }}

restore-keys: |

Library-

- uses: game-ci/unity-test-runner@v2

env:

UNITY_LICENSE: \${{ secrets.UNITY_LICENSE }}

with:

testMode: \${{ matrix.testMode }}

artifactsPath: \${{ matrix.testMode }}-artifacts

```
- uses: actions/upload-artifact@v2  
if: always()  
with:  
  name: Test-${{ matrix.testMode }}  
  path: ${{ matrix.testMode }}-artifacts
```

8. Testing and Debugging

8.1 Unit Tests

8.1.1 Test Structure

We use the Unity Test Framework to create unit tests:

```
csharp
```

```
using System.Collections;
```

```
using NUnit.Framework;
```

```
using UnityEngine;
```

```
using UnityEngine.TestTools;
```

```
namespace Tests
```

```
{
```

```
    public class PlayerControllerTests
```

```
    {
```

```
        private GameObject _playerObject;
```

```
        private PlayerController _controller;
```

```
        [SetUp]
```

```
        public void Setup()
```

```
        {
```

```
            // Create a test player
```

```
            _playerObject = new GameObject("TestPlayer");
```

```
            _controller = _playerObject.AddComponent<PlayerController>();
```

```
            // Configure dependencies for tests
```

```
// ...  
}
```

[TearDown]

```
public void Teardown()  
{  
    // Clean up after each test  
    Object.Destroy(_playerObject);  
}
```

[Test]

```
public void PlayerHealth_TakeDamage_ReducesHealth()  
{  
    // Arrange  
    float initialHealth = 100f;  
    float damageAmount = 25f;  
    _controller.SetHealth(initialHealth);  
  
    // Act  
    _controller.TakeDamage(damageAmount);  
  
    // Assert  
    Assert.AreEqual(initialHealth - damageAmount, _controller.Health);  
}
```

[UnityTest]

```
public IEnumerator PlayerMovement_Jump_IncreasesYPosition()  
{  
    // Arrange  
    Vector3 initialPosition = _playerObject.transform.position;
```

```

        // Act
        _controller.Jump();

        // Wait for physics
        yield return new WaitForSeconds(0.1f);

        // Assert
        Assert.Greater(_playerObject.transform.position.y, initialPosition.y);
    }
}

```

8.1.2 Mocking and Test Doubles

For testing components with external dependencies:

csharp

```

public interface IInputService
{
    bool IsJumpPressed();
    Vector2 GetMovementInput();
}

```

// Real implementation

```

public class PlayerInputService : MonoBehaviour, IInputService
{
    public bool IsJumpPressed()
    {
        return Input.GetButtonDown("Jump");
    }

    public Vector2 GetMovementInput()
    {
        return new Vector2(Input.GetAxis("Horizontal"), 0);
    }
}

```



```
}  
}
```

```
// Mock for tests
```

```
public class MockInputService : IInputService  
{  
    public bool JumpPressed { get; set; }  
    public Vector2 MovementInput { get; set; }  
  
    public bool IsJumpPressed() => JumpPressed;  
    public Vector2 GetMovementInput() => MovementInput;  
}
```

```
// Test using the mock
```

```
[Test]  
public void PlayerController_JumpInput_CallsJumpMethod()  
{  
    // Arrange  
    var mockInput = new MockInputService { JumpPressed = true };  
    _controller.SetInputService(mockInput);  
  
    // Observe behavior (via the spy)  
    bool jumpMethodCalled = false;  
    _controller.OnJump += () => jumpMethodCalled = true;  
  
    // Act  
    _controller.ProcessInput();  
  
    // Assert  
    Assert.IsTrue(jumpMethodCalled);  
}
```

8.2 Debugging Tools

8.2.1 Advanced Logging System

csharp

```
public static class GaiaLogger
```

```
{
```

```
    public enum LogLevel
```

```
    {
```

```
        Debug,
```

```
        Info,
```

```
        Warning,
```

```
        Error,
```

```
        Fatal
```

```
    }
```

```
    private static LogLevel _currentLogLevel = LogLevel.Debug;
```

```
    public static void SetLogLevel(LogLevel level)
```

```
    {
```

```
        _currentLogLevel = level;
```

```
    }
```

```
    public static void Log(LogLevel level, string message, Object context = null)
```

```
    {
```

```
        if (level < _currentLogLevel)
```

```
            return;
```

```
        string formattedMessage = $"[{level}] {message}";
```

```
        switch (level)
```

```
        {
```

```
            case LogLevel.Debug:
```

```

    case LogLevel.Info:
        Debug.Log(formattedMessage, context);
        break;
    case LogLevel.Warning:
        Debug.LogWarning(formattedMessage, context);
        break;
    case LogLevel.Error:
    case LogLevel.Fatal:
        Debug.LogError(formattedMessage, context);
        break;
}

// Optional: logging to file
LogToFile(level, message);
}

private static void LogToFile(LogLevel level, string message)
{
    #if !UNITY_EDITOR && !DEBUG
        // Logging to file only in builds
        // Implementation...
    #endif
}

// Convenience methods

public static void Debug(string message, Object context = null) => Log(LogLevel.Debug,
message, context);

public static void Info(string message, Object context = null) => Log(LogLevel.Info, message,
context);

public static void Warning(string message, Object context = null) => Log(LogLevel.Warning,
message, context);

```

```

    public static void Error(string message, Object context = null) => Log(LogLevel.Error, message,
context);

    public static void Fatal(string message, Object context = null) => Log(LogLevel.Fatal, message,
context);
}

```

8.2.2 In-Game Visualization Tools

csharp

```

public class DebugVisualizer : MonoBehaviour
{
    [SerializeField] private bool _showDebugInfo = false;

    [Header("Display Options")]
    [SerializeField] private bool _showPlayerStats = true;
    [SerializeField] private bool _showFPS = true;
    [SerializeField] private bool _showMemoryUsage = true;
    [SerializeField] private bool _showColliders = false;

    private PlayerController _player;
    private float _deltaTime = 0.0f;
    private GUIStyle _guiStyle = new GUIStyle();

    private void Start()
    {
        _player = FindObjectOfType<PlayerController>();

        // Text style
        _guiStyle.fontSize = 20;
        _guiStyle.normal.textColor = Color.white;
        _guiStyle.fontStyle = FontStyle.Bold;
    }

    private void Update()

```

```

{
    _deltaTime += (Time.unscaledDeltaTime - _deltaTime) * 0.1f;

    // Toggle with a key
    if (Input.GetKeyDown(KeyCode.F3))
    {
        _showDebugInfo = !_showDebugInfo;
    }
}

private void OnGUI()
{
    if (!_showDebugInfo)
        return;

    int y = 10;

    if (_showFPS)
    {
        float fps = 1.0f / _deltaTime;

        GUI.Label(new Rect(10, y, 200, 20), $"FPS: {Mathf.Round(fps)}", _guiStyle);

        y += 25;
    }

    if (_showMemoryUsage)
    {
        float memoryUsage = System.GC.GetTotalMemory(false) / (1024f * 1024f);

        GUI.Label(new Rect(10, y, 300, 20), $"Memory: {memoryUsage:F2} MB", _guiStyle);

        y += 25;
    }
}

```

```

if (_showPlayerStats && _player != null)
{
    GUI.Label(new Rect(10, y, 300, 20), $"Health: {_player.Health:F0}/{_player.MaxHealth:F0}",
_guiStyle);
    y += 25;
    GUI.Label(new Rect(10, y, 300, 20), $"Energy: {_player.Energy:F0}/{_player.MaxEnergy:F0}",
_guiStyle);
    y += 25;
    GUI.Label(new Rect(10, y, 300, 20), $"Position: {_player.transform.position}", _guiStyle);
    y += 25;
}
}

```

```

private void OnDrawGizmos()

```

```

{
    if (!_showDebugInfo || !_showColliders)
        return;

```

```

    // Visualize colliders

```

```

    Collider2D[] colliders = FindObjectsOfType<Collider2D>();

```

```

    foreach (Collider2D collider in colliders)

```

```

    {
        Gizmos.color = Color.green;

```

```

        if (collider is BoxCollider2D)

```

```

        {
            BoxCollider2D boxCollider = collider as BoxCollider2D;
            Gizmos.matrix = boxCollider.transform.localToWorldMatrix;
            Gizmos.DrawWireCube(boxCollider.offset, boxCollider.size);
        }

```

```

        else if (collider is CircleCollider2D)

```

```

        {

```

```

        CircleCollider2D circleCollider = collider as CircleCollider2D;

        Gizmos.matrix = circleCollider.transform.localToWorldMatrix;

        Gizmos.DrawWireSphere(circleCollider.offset, circleCollider.radius);
    }
}
}
}

```

9. Reference API

9.1 Main Game System

9.1.1 GameManager

The GameManager is the central control point of the game, managing global state and system coordination.

Methods:

csharp

// Initialize the game with specified parameters

```
public void Initialize(GameSettings settings)
```

// Start a new game

```
public void StartNewGame()
```

// Load a saved game

```
public bool LoadGame(int saveSlot)
```

// Save the current game

```
public void SaveGame(int saveSlot)
```

// Pause the game

```
public void PauseGame(bool isPaused)
```

// Change the current level

```
public void ChangeLevel(string levelName, Vector2 spawnPosition)
```

// Handle player death

```
public void HandlePlayerDeath()
```

// Reset player to last checkpoint

```
public void RespawnPlayerAtCheckpoint()
```

Properties:

csharp

// Current game state

```
public GameState CurrentGameState { get; private set; }
```

// Current game level

```
public LevelManager CurrentLevel { get; private set; }
```

// Reference to player controller

```
public PlayerController Player { get; private set; }
```

// Indicates if game is paused

```
public bool IsPaused { get; private set; }
```

// Global purification level (0-100)

```
public float GlobalPurificationLevel { get; private set; }
```

9.1.2 PlayerController

The PlayerController manages all actions and state of the main character.

Methods:

csharp

// Apply damage to the player

```
public void TakeDamage(float amount, Vector2 knockbackDirection)
```

// Restore health

```
public void Heal(float amount)
```


// Make the character jump

```
public void Jump()
```

// Perform a dash in the specified direction

```
public void Dash(Vector2 direction)
```

// Activate a melee attack

```
public void MeleeAttack()
```

// Activate a ranged attack

```
public void RangedAttack()
```

// Activate a special ability

```
public void ActivateAbility(AbilityType abilityType)
```

// Interact with the object in front of the player

```
public void Interact()
```

// Purify an area

```
public void PurifyArea(PurificationNode node)
```

Properties:

csharp

// Current and maximum health

```
public float Health { get; private set; }
```

```
public float MaxHealth { get; private set; }
```

// Current and maximum energy

```
public float Energy { get; private set; }
```

```
public float MaxEnergy { get; private set; }
```

// Current player state

```
public PlayerState CurrentState { get; private set; }
```

// Current movement direction

```
public Vector2 MoveDirection { get; private set; }
```

// Current movement speed

```
public float CurrentSpeed { get; private set; }
```

// Indicates if player is grounded

```
public bool IsGrounded { get; private set; }
```

// Unlocked abilities

```
public HashSet<AbilityType> UnlockedAbilities { get; private set; }
```

9.2 Enemy System

9.2.1 EnemyController

Base class for all enemy types.

Methods:

csharp

// Initialize the enemy with specified parameters

```
public virtual void Initialize(EnemySettings settings)
```

// Apply damage to the enemy

```
public virtual void TakeDamage(float amount, Vector2 hitDirection)
```

// Activate attack behavior

```
public virtual void Attack()
```

// Temporarily stun the enemy

```
public virtual void Stun(float duration)
```

```

// Apply purification weakening effect
public virtual void ApplyPurificationEffect(float purificationLevel)

// Move the enemy towards target position
public virtual void MoveTowards(Vector2 targetPosition, float speed)

Properties:

csharp
// Current and maximum health
public float Health { get; protected set; }
public float MaxHealth { get; protected set; }

// Current enemy state
public EnemyState CurrentState { get; protected set; }

// Enemy type
public EnemyType Type { get; protected set; }

// Combat statistics
public float AttackDamage { get; protected set; }
public float AttackRange { get; protected set; }
public float AttackSpeed { get; protected set; }

// Player perception
public bool CanSeePlayer { get; protected set; }
public Vector2 LastKnownPlayerPosition { get; protected set; }

```

9.3 Environment System

9.3.1 LevelManager

Manages the state and features of a level.

Methods:

```

csharp
// Initialize the level

```

```
public void Initialize()
```

```
// Load all level elements
```

```
public void LoadLevel()
```

```
// Unload the level and free resources
```

```
public void UnloadLevel()
```

```
// Activate a checkpoint
```

```
public void ActivateCheckpoint(string checkpointId)
```

```
// Purify a zone of the level
```

```
public void PurifyZone(string zoneId, float purificationAmount)
```

```
// Spawn an enemy at the specified position
```

```
public GameObject SpawnEnemy(EnemyType type, Vector2 position)
```

```
// Open a passage to another level
```

```
public void OpenLevelTransition(string targetLevelId, string entryPointId)
```

```
// Unlock an ability for the player
```

```
public void UnlockAbility(AbilityType abilityType)
```

```
Properties:
```

```
csharp
```

```
// Unique level identifier
```

```
public string LevelId { get; private set; }
```

```
// Current level state
```

```
public LevelState CurrentState { get; private set; }
```

```
// Active checkpoint
```

```
public Checkpoint ActiveCheckpoint { get; private set; }
```

```
// Global level purification level (0-100)
```

```
public float PurificationLevel { get; private set; }
```

```
// Level purification zones
```

```
public Dictionary<string, PurificationZone> PurificationZones { get; private set; }
```

```
// Found collectibles
```

```
public List<string> FoundCollectibles { get; private set; }
```

9.3.2 PurificationSystem

Manages environment purification and its effects.

Methods:

csharp

```
// Initialize the system with level zones
```

```
public void Initialize(List<PurificationZone> zones)
```

```
// Purify a specific zone
```

```
public void PurifyZone(string zoneId, float amount)
```

```
// Update the visual state of the environment based on purification level
```

```
public void UpdateEnvironmentVisuals()
```

```
// Apply purification effects to enemies in a zone
```

```
public void ApplyPurificationEffects(string zoneId)
```

```
// Check if all zones are purified
```

```
public bool IsLevelFullyPurified()
```

```
// Reset the system for a new level
```

```
public void Reset()
```

Properties:

csharp

// Global purification level (0-100)

```
public float GlobalPurificationLevel { get; private set; }
```

// Purification state by zone

```
public Dictionary<string, float> ZonePurificationLevels { get; private set; }
```

// Purification events

```
public event Action<float> OnGlobalPurificationChanged;
```

```
public event Action<string, float> OnZonePurificationChanged;
```

10. Appendices

10.1 UML Diagrams

Complete UML diagrams are available in the following files:

- Diagrams/ClassDiagram.png - General class structure
- Diagrams/StateChart.png - State diagram for player and enemies
- Diagrams/SequenceDiagram.png - Interaction flow between main systems

10.2 Developer Control Cheatsheet

Player Controls (default configuration)

- WASD / Arrow keys - Movement
- Space - Jump (double jump if pressed twice)
- Shift - Air dash
- E - Interaction
- Left click - Melee attack
- Right click - Ranged attack (Pollen Vortex)
- 1-4 - Power selection
- F - Use selected power
- C - Camouflage
- Ctrl - Slide

Debug Commands (developer mode only)

- F1 - Toggle God mode (invincibility)

- F2 - Toggle collisions
- F3 - Show/hide debug information
- F4 - Reload current level
- F5 - Quick save
- F6 - Load last save
- F7 - Kill all enemies in the level
- F8 - Unlock all abilities
- F9 - Teleport to next checkpoint
- F10 - Instant purification of current zone

10.3 Common Problem Resolution

Problem	Possible Cause	Solution
Freezes during level transitions	Synchronous resource loading	Convert to asynchronous loading with <code>LoadSceneAsync</code>
Memory leaks with particles	Undeleted particles	Use the pooling system and check references
Slowdowns with numerous enemies	Too many simultaneous physics calculations	Implement a LOD system for physics
Animations locking up	Incorrect transitions in Animator	Check transition conditions and parameters
Ghost collisions	Hitbox or layer problem	Check collision matrix and collider sizes

10.4 Technical Term Glossary

- **FSM (Finite State Machine):** State management system for entity behavior
- **LOD (Level of Detail):** Optimization technique reducing the complexity of distant objects
- **Pooling:** Object reuse to avoid costly instantiation/destruction operations
- **Raycasting:** Detection technique using "rays" to identify objects
- **Hitbox:** Collision zone used for hit detection
- **Sprite Atlas:** Texture grouping to optimize rendering
- **Coroutine:** Function that can be suspended and resumed, used for asynchronous operations
- **Tilemap:** Level creation system based on tiles
- **Shader:** Program executed on the GPU for graphic rendering

- **Prefab:** Reusable object template in Unity